

Transformers

1. Transformers can [2]:
 1. process an unordered set of tokens/vectors
 1. Positional encoding adds spatial information.
 2. self-attention is like clustering and replacing vectors with centers, except
 1. Multiple, complex, learnable similarity functions.
 2. No need to preset number of clusters.
 3. Vectors are iteratively aggregated and transformed.
 4. Vectors can start as purely local information (e.g. 16x16 patch).
2. What kind of word embedding is used in the original transformer?
 1. Recall example of word embeddings such as one-hot, word2vec, GloVe, etc.
 2. There are *two options* for dealing with the Pytorch nn.Embedding weight matrix: [3]
 1. Initialize it with pre-trained embeddings and keep it fixed, in which case it's really just a lookup table.
 2. Initialize it randomly, or with pre-trained embeddings, but keep it trainable. In that case the word representations will get refined and modified throughout training because the weight matrix will get refined and modified throughout training. The transformer uses a random initialization of the weight matrix and refines these weights during training - i.e. it learns its own word embeddings. Note: Recall in backpropagation we can get $\frac{\partial L}{\partial \vec{x}}$.
3. Transformer decoder [4]
 1. in NMT the partially decoded sentence is used as query in the first multi-head attention block.
 2. the need for decoder
 1. The decoder is used when the output of the model is a sequence.
 2. If the output of the model is single value, e.g. for a classification task or a single-value regression task, the encoder would suffice.
 3. When predicting multiple values of a time series, a decoder should probably be used that let to condition not only on the input, but also on the partially generated output values.

References:

1. Formal Algorithms for Transformers, Mary Phuong, Marcus Hutter.
2. CS598, Derek Hoesim, UIUC.
3. [The Transformer: Attention Is All You Need](#)
4. [Role of decoder in transformer](#)